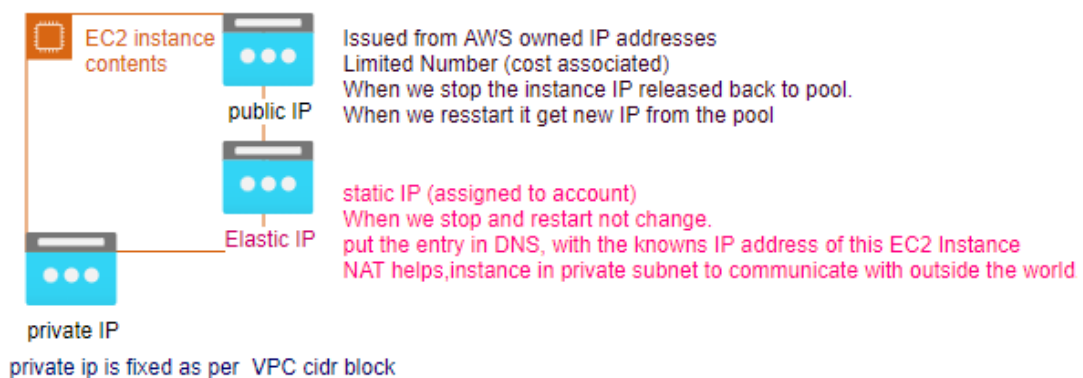
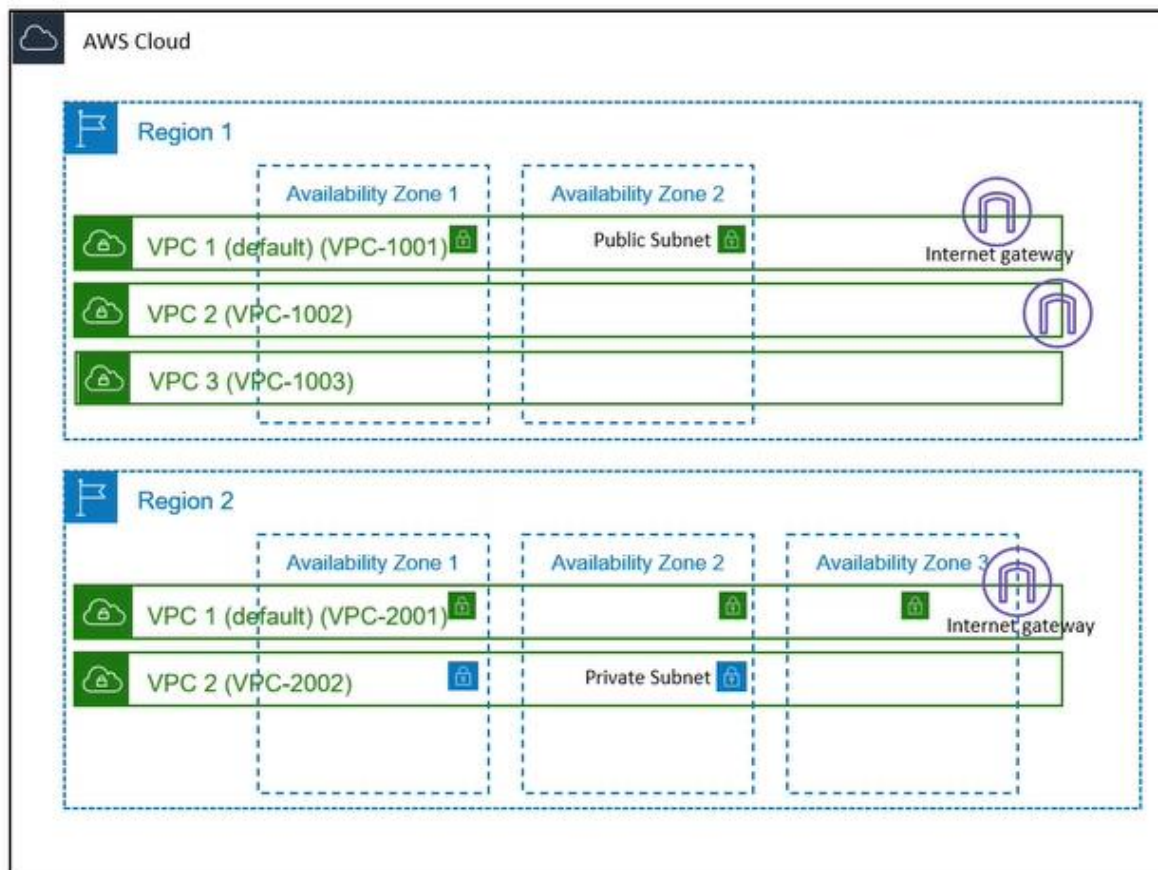
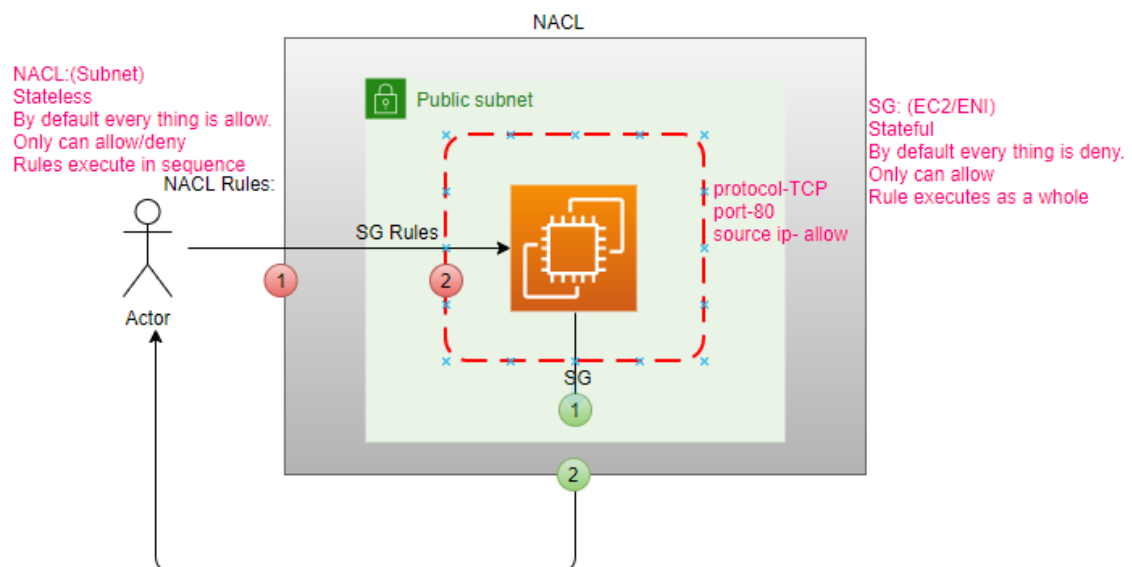
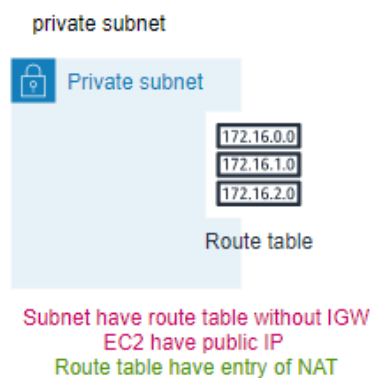
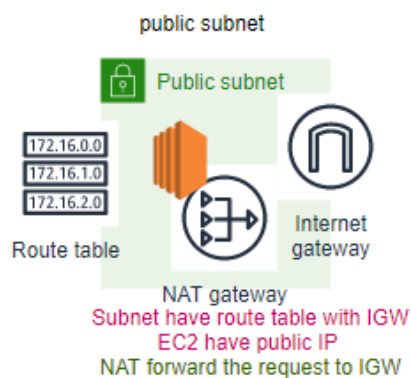
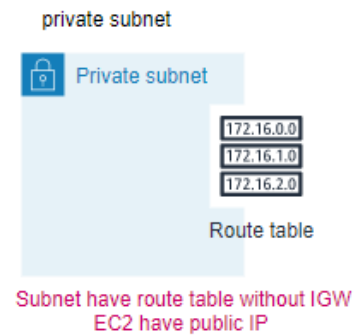
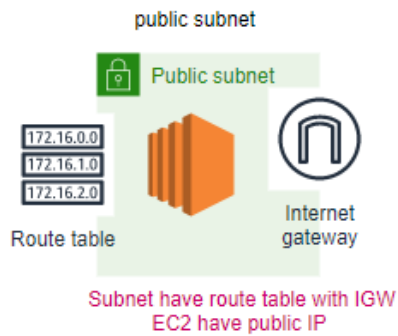


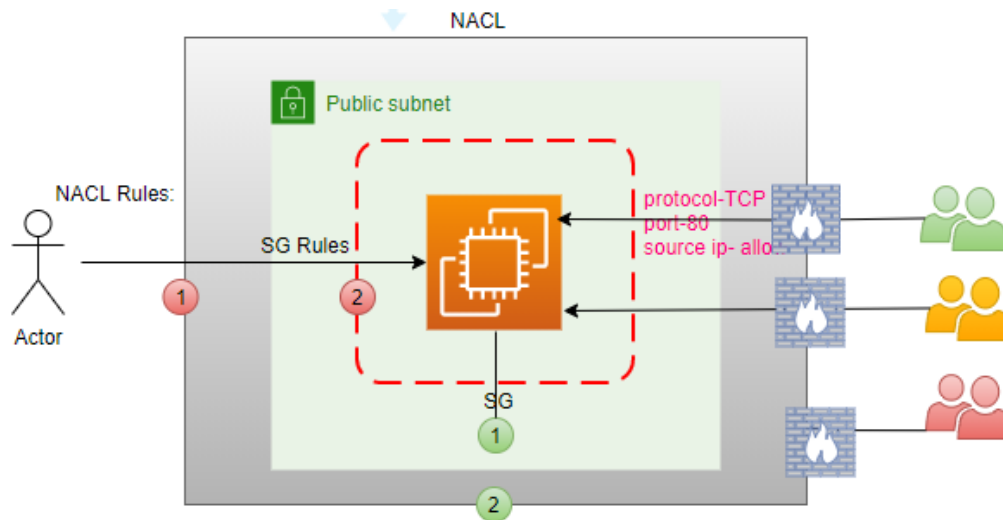
Day2

VPC (IGW, main route table), AZ, Subnet (route table) & IP Address.





Why we need NACL?



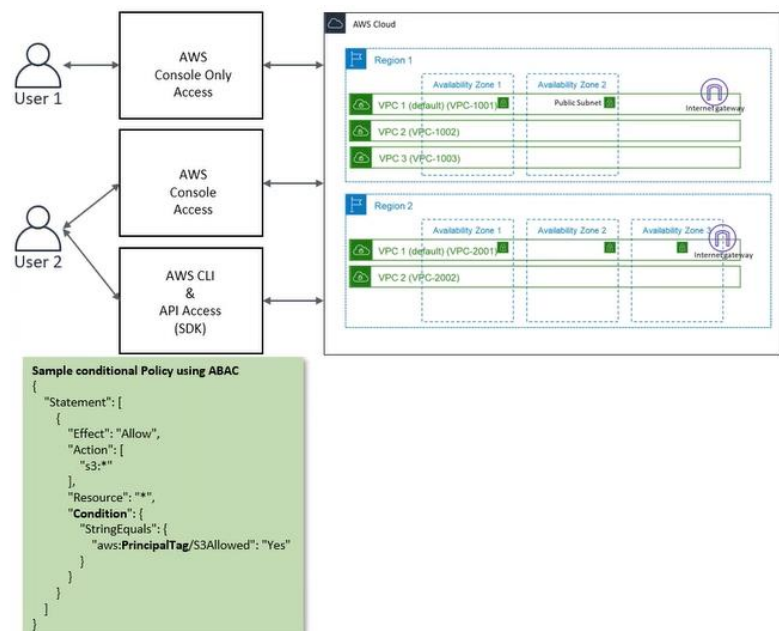
IAM: (who has access to the resources)

IAM (Authorization, Authentication, Users, Groups, Roles, Policy)

ABAC (Attribute based access control): Using tags

Context: With in the Account

- Access Modes
 - Console Only
 - Console + Programmatic
- IAM - [Identity and Access Management](#)
 - Authentication
 - Authorization
 - Users
 - Groups
 - Password Policy
 - Multifactor Authentication
- [Billing Access](#)
- Tags and Usage (ABAC)



step1: IAM-> create a user

Add user

1 2 3 4 5

Set user details

You can add multiple users at once with the same access type and permissions. [Learn more](#)

User name*

[+ Add another user](#)

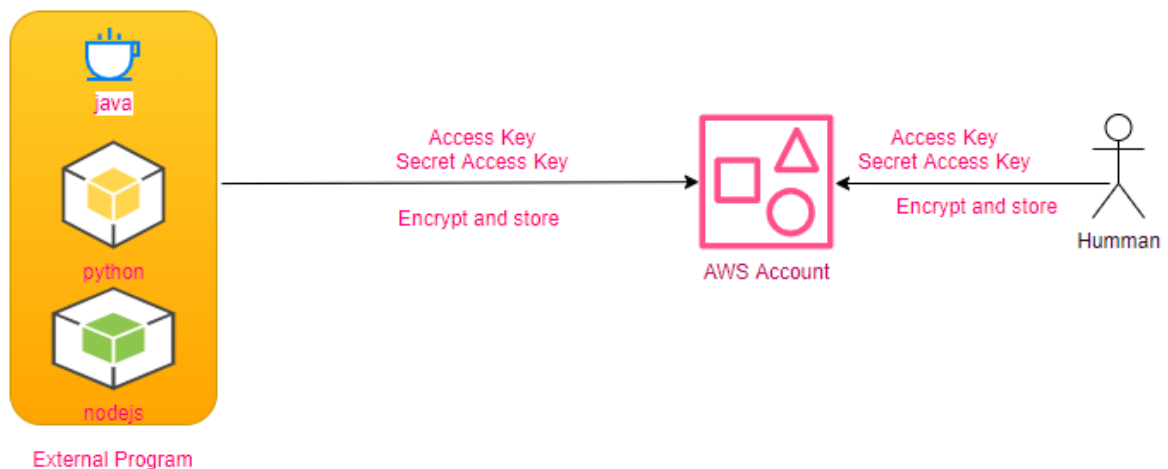
Select AWS access type

Select how these users will access AWS. Access keys and autogenerated passwords are provided in the last step. [Learn more](#)

- Access type* ☐ **Programmatic access**
Enables an **access key ID** and **secret access key** for the AWS API, CLI, SDK, and other development tools.
- ☐ **AWS Management Console access**
Enables a **password** that allows users to sign-in to the AWS Management Console.

Access Modes:

- Programmatic(SDK,CLI): access key id, secret access key
- AWS Console: password



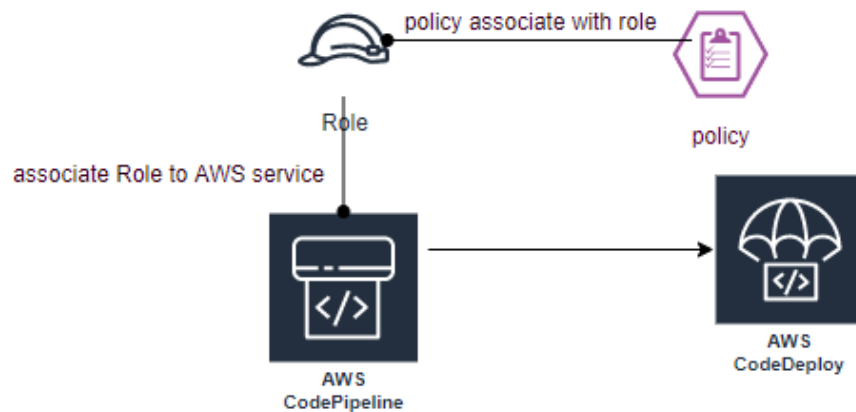
User Group: One than more Users

Policy: access permissions:

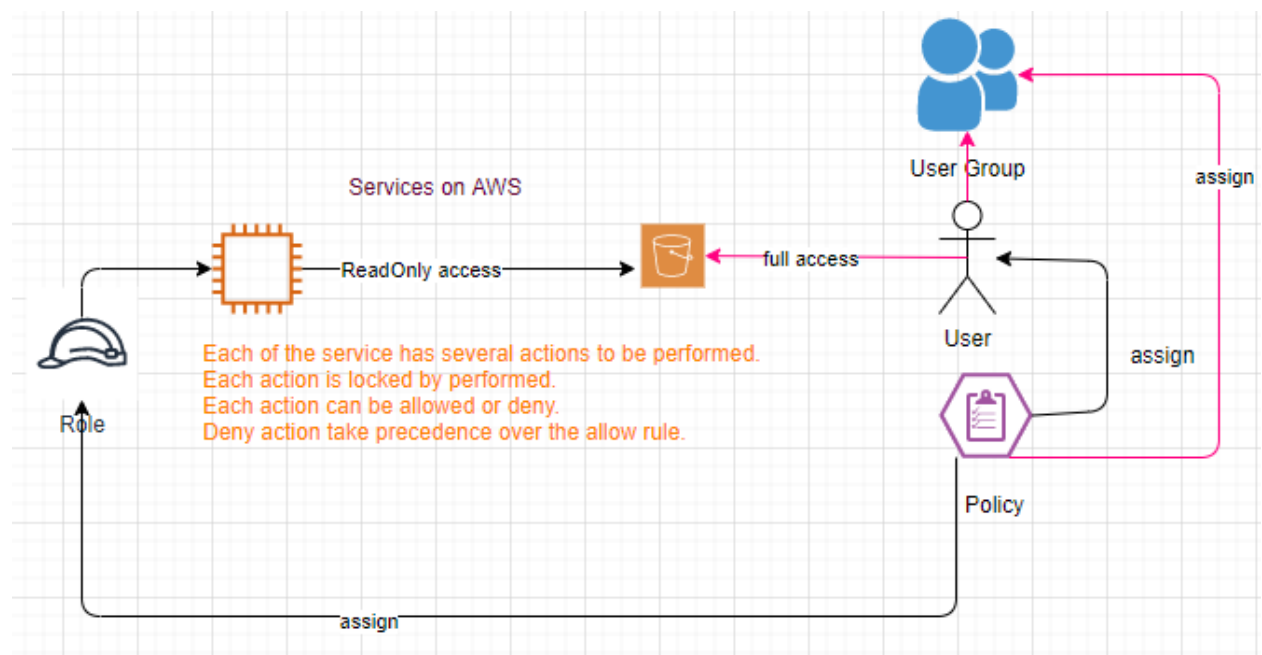
```
"Statements": { "Effect": "Allow",
                 "Action": ["s3-outposts:Get*", "s3-outposts:List*"],
                 "Resource": "*",
                 "Condition": {tag} },]
```

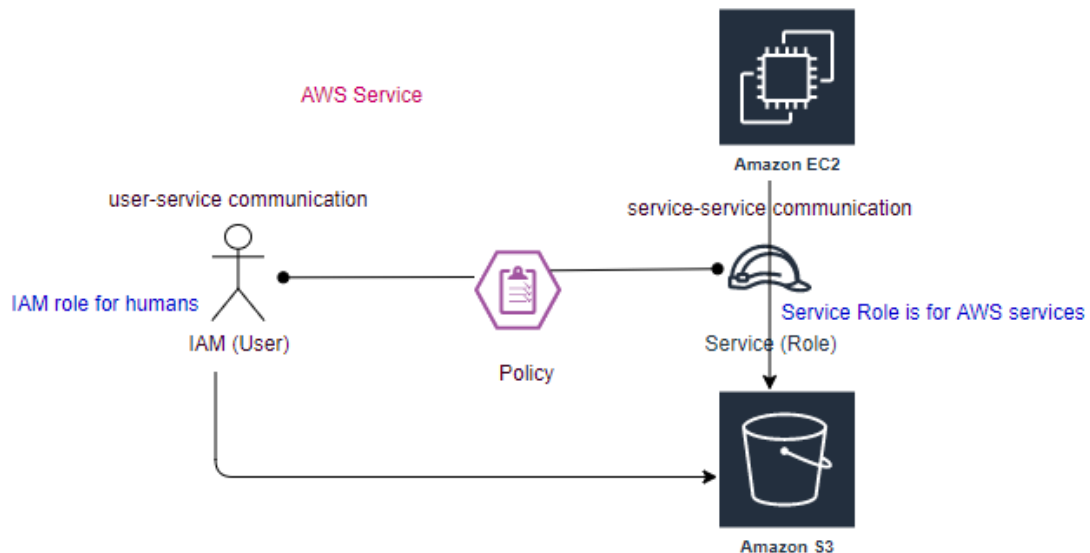
Roles: (Initiater of the Role)

Two services (AWS resources) with in AWS must be communicate to each other through the Role.



When **human** trying to access AWS resources using: console / Access and Secret Key, when **AWS resource** trying to access another AWS resource using: role, **Policies** can be assigned to user/user group **or to Role**.





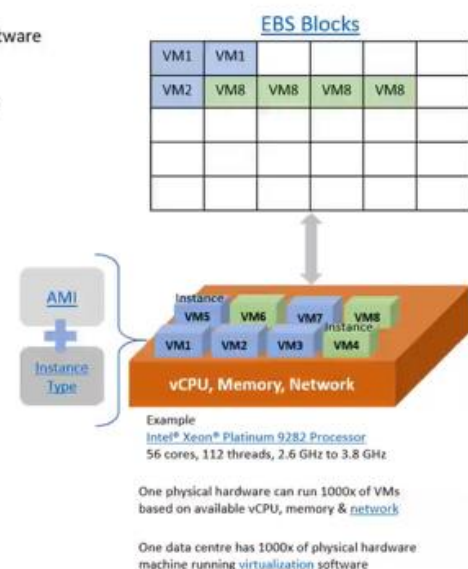
access key and secret key stored in .aws\credentials its only for CLI and SDK access. Its not for console access, its for external access.

Policies are written in JSON (Java Script Object Notation).

Each service can provide different actions using policies can allow or deny specific action.

EC2: (AMI)

- **AMI**
 - Amazon Machine Image. Preconfigured templates of an OS + additional software
- **Instance Type**
 - Various configuration for of CPU, memory, storage and networking capacity
- **Instance**
 - VM configured for computing purpose using AMI + Instance Type
 - [Instance State](#)
- **EBS**
 - Elastic Block Storage, persistent storage volume for storing data
- **Instance Store**
 - Temporary data store, deleted when an instance is stopped or terminated
- **IPv4 address**
 - Private, Public and Elastic
- **[Security Group](#)**
 - Instance firewall, network defence layer along with subnet level NACL
- **[User Data](#)**
 - Set of commands executed on an EC2 instance when it is launched



AMI: O/S + additional s/w

- Root Device Type (boot drive): ebs (persistent)/ instance store(temprary)
- Virtulization Type: hvm/paravirtual
- ENA enabled: Yes/No

Instance Type: Various configuration for CPU, memory, storage and network capacity.

Instance Store: Temporary (transitory work load)

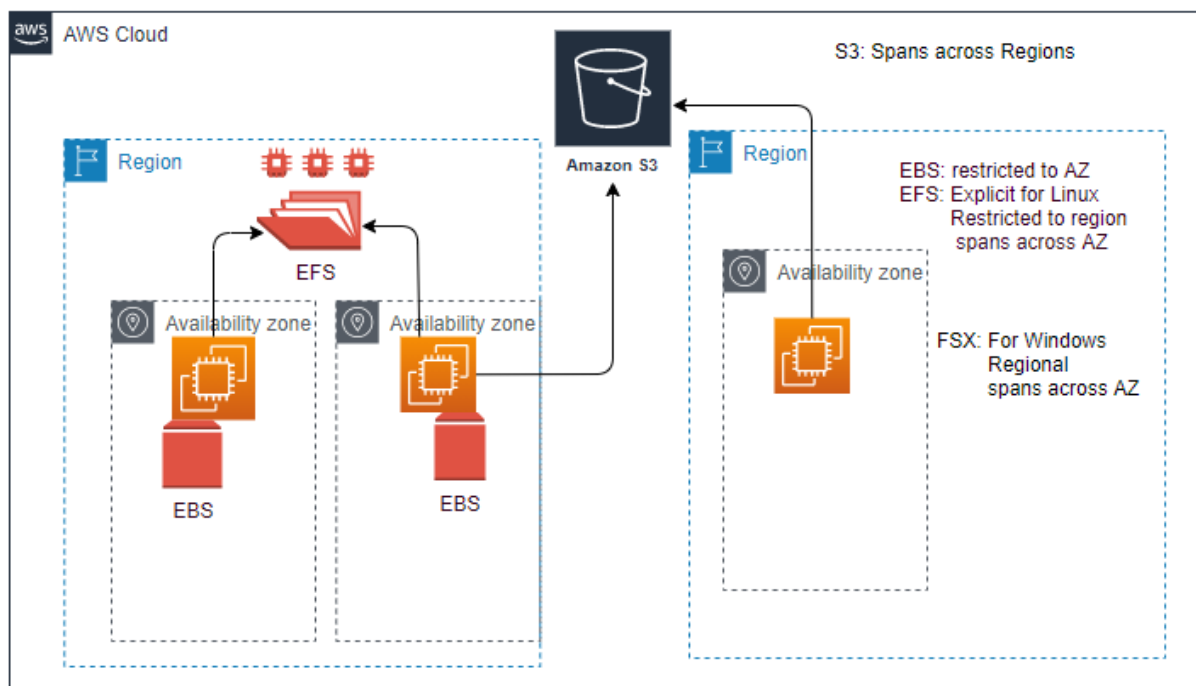
EBS: (Elastic Block Storage): Can Only Increase the Storage, but can't reduce, have boot drive.

Logical Unit Number (LUN)

Difference between EBS enabled (storage) Instance Store enabled devices

O/S has to load from some where, It is a temporary drive as soon as we stop the device instance type drive is revoked.

Storage Type: EBS (persistent volume to store data), EFS, S3(Object Storage)



Status Checks

System Check: This check verifies that your instance is reachable. We test that we are able to get network packets to your instance. If this check fails, there may be an issue with the infrastructure hosting your instance (such as AWS power, networking or software system). You may need to restart or replace the instance, wait for our system to resolve the issue, or seek technical support. This check does not validate that your operating system and applications are accepting traffic. (Underneath configuration is available and not corrupted)

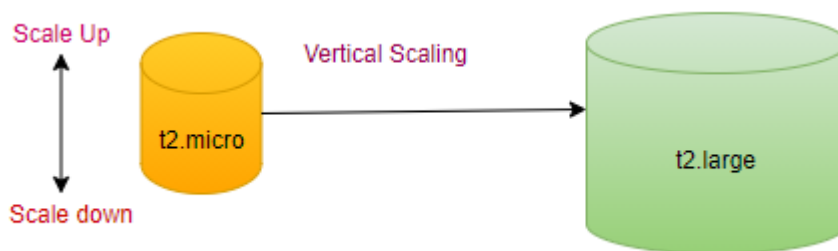
Instance Status Check: Once System check is passed, this check verifies that your instance's O/S is accepting traffic. If this check fails you may need to reboot your instance or make modification to your operating system configuration. (Is reachable from outside the world)



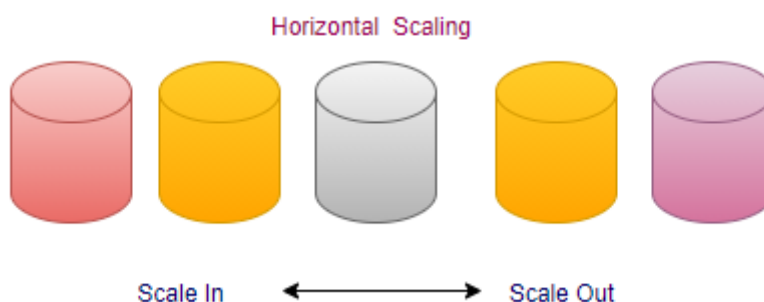
Scaling:

Vertical Saling: Stop (update from t2.micro->t2.small) , and then restart.

Can't vertical scale instance type at run time, first stop the instance then vertical scale the type.



Horizontal Saling: automate with zero downtime, Increase the number of resources of same capacity.



User Data:

Set of commands executed on the EC2 instance when it is launched (boot starp).

step1: choose the EC2 instance.

step1.1: right click-> Monitor and troubleshoot-> Get System log

All communication within subnet through private IP.

HA through multi-AZ.

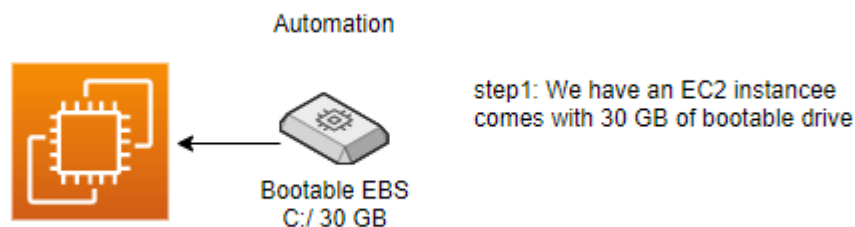
AWS CLI:

```
S:\>aws configure
AWS Access Key ID [*****WIY5]:
AWS Secret Access Key [*****Kcsc]:
Default region name [ap-south-1]:
Default output format [json]:
```

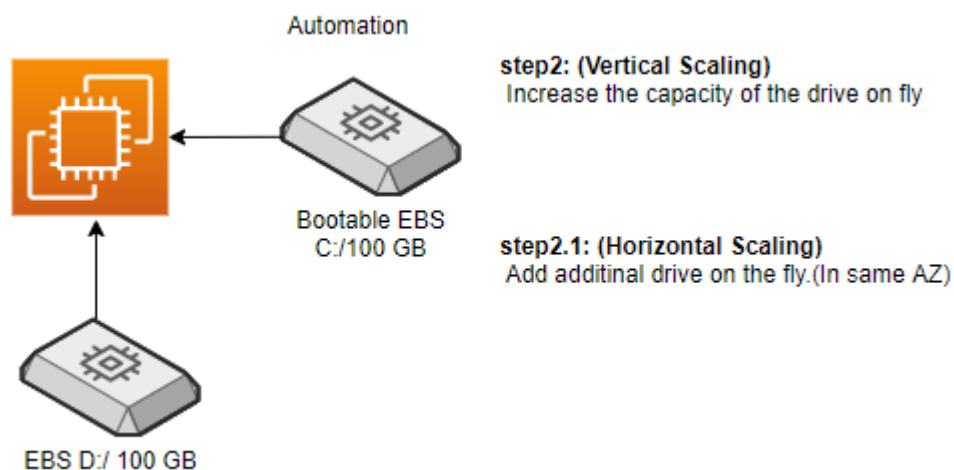
aws s3 ?

aws s3 cp ?

Automate:



UC: Can we make it 100 GB on the fly?

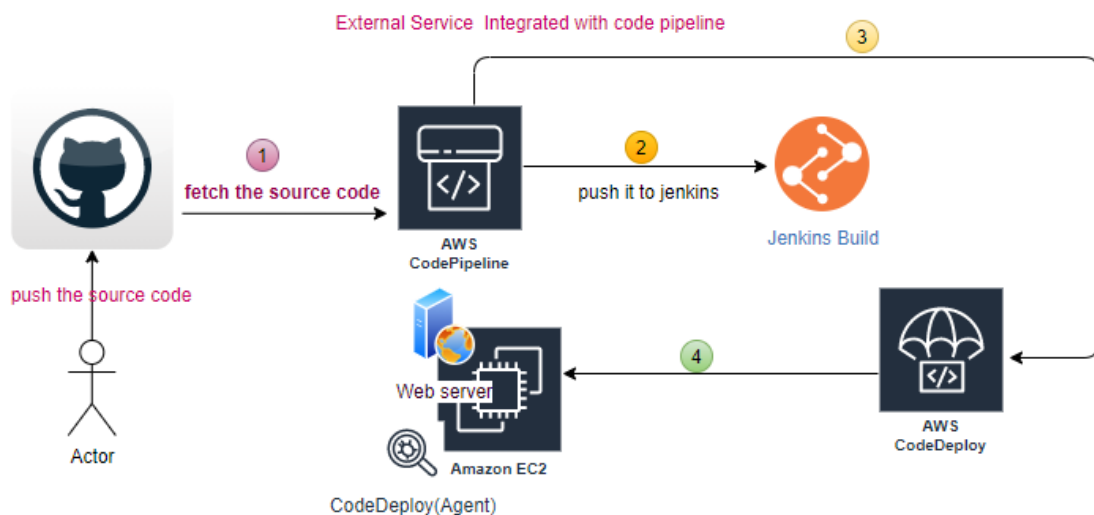
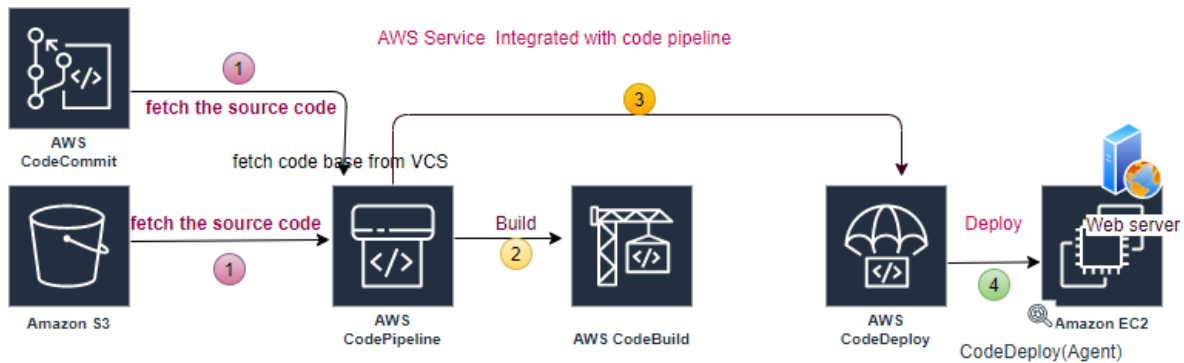


Only can increase the size, can't decrease. Take small resource, test and increase capacity if required.

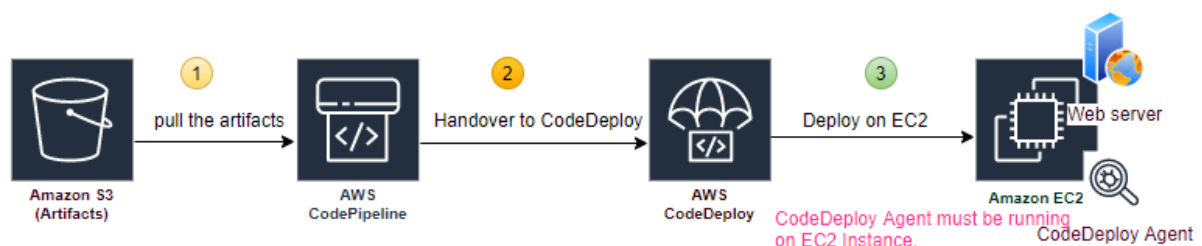
AWS DevOps:

DevOps is a culture where developers and operation teams work together.

Code Pipeline: Carrying out certain tasks in sequence(`appspec.yaml`).

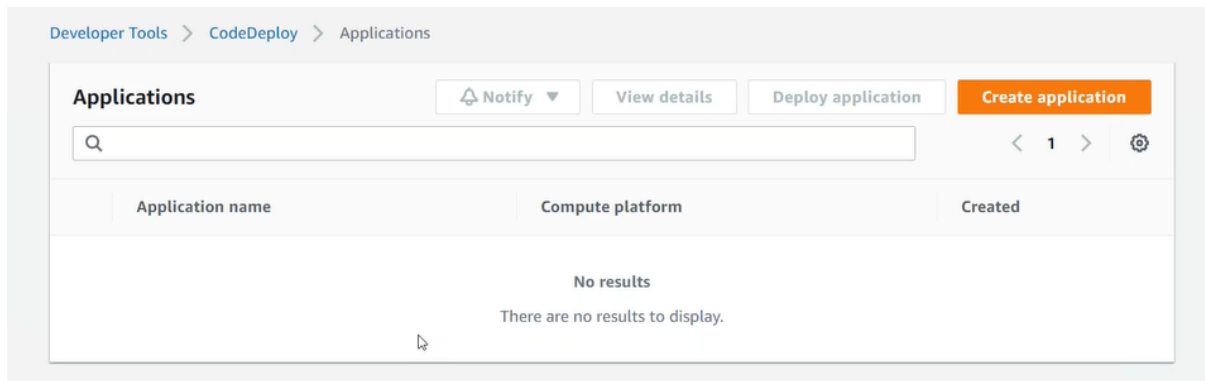


UC1: pick from s3 and deploy toward my webserver (EC2)

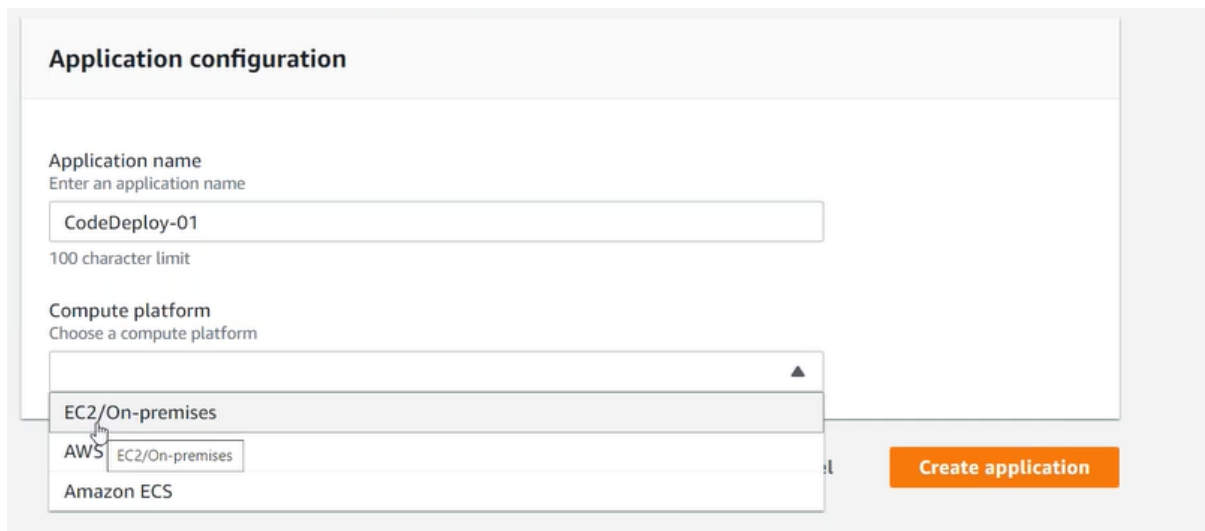


step1: Install Webserver and CodeDeploy agent on EC2 (tag).

step2: create an application in CodeDeploy.



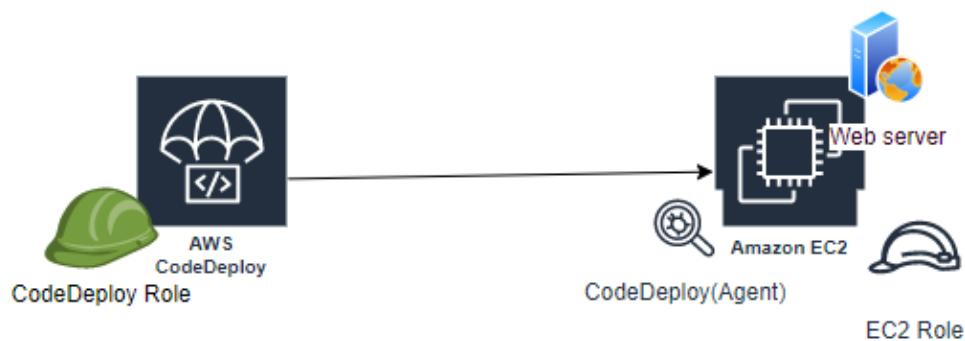
step2.1: Application Configuration.



step2.2: Create Deployment Groups (Group of EC2 instances).

step2.3: Service Role (CodeDeploy access to your target Instances), CodeDeploy communicates with CodeDeployment Agent running on EC2 Instance. Then Agent follow the Instruction (script appspec.yaml).

step2.4: Agent also has to communicate back to CodeDeploy, EC2 must have role assigned to it, to communicate with CodeDeploy.



step3: Go to pipeline (ensure source code is available on S3, Instance are running and CodeDeploy agent running on it).

Choose pipeline settings [Info](#)

Pipeline settings

Pipeline name
Enter the pipeline name. You cannot edit the pipeline name after it is created.

No more than 100 characters

Service role

☒ **New service role**
Create a service role in your account

☐ **Existing service role**
Choose an existing service role from your account

Role name

Type your service role name

☒ Allow AWS CodePipeline to create a service role so it can be used with this new pipeline

step3.1: Add source stage.

Add source stage [Info](#)

Source

Source provider
This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.

Q |

AWS CodeCommit

Amazon ECR

Amazon S3

Bitbucket

GitHub (Version 1)

GitHub (Version 2)

GitHub Enterprise Server

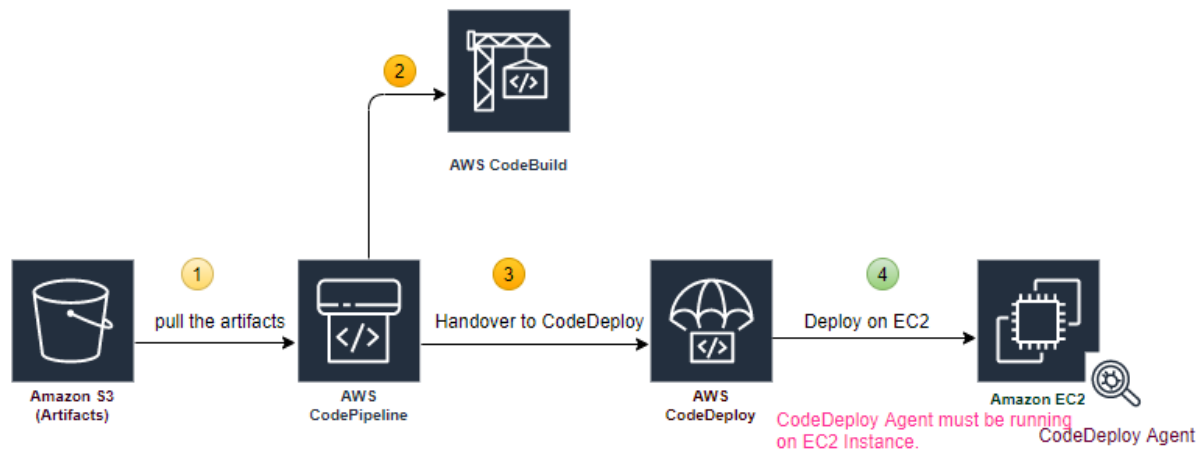
Previous

Next

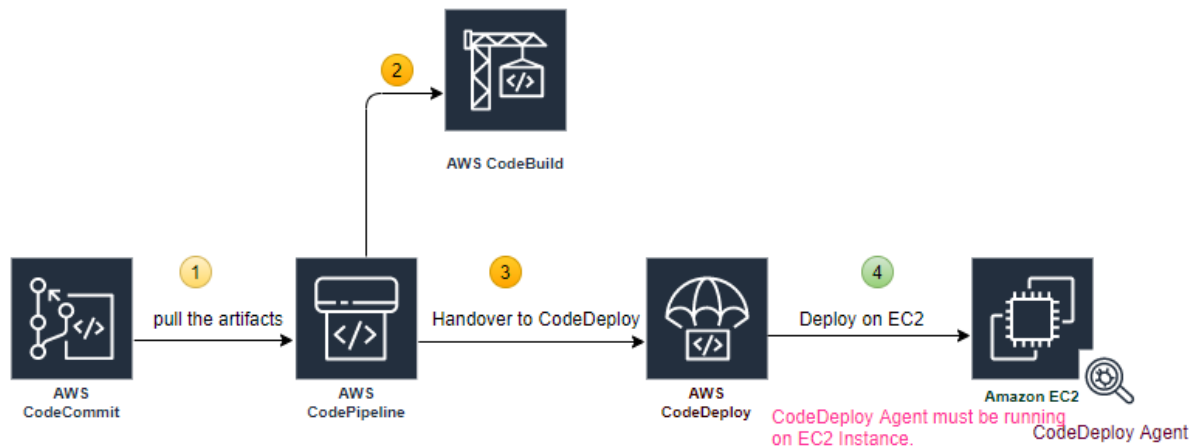
step3.2: Add build stage (optional).

step3.3: Add deploy stage

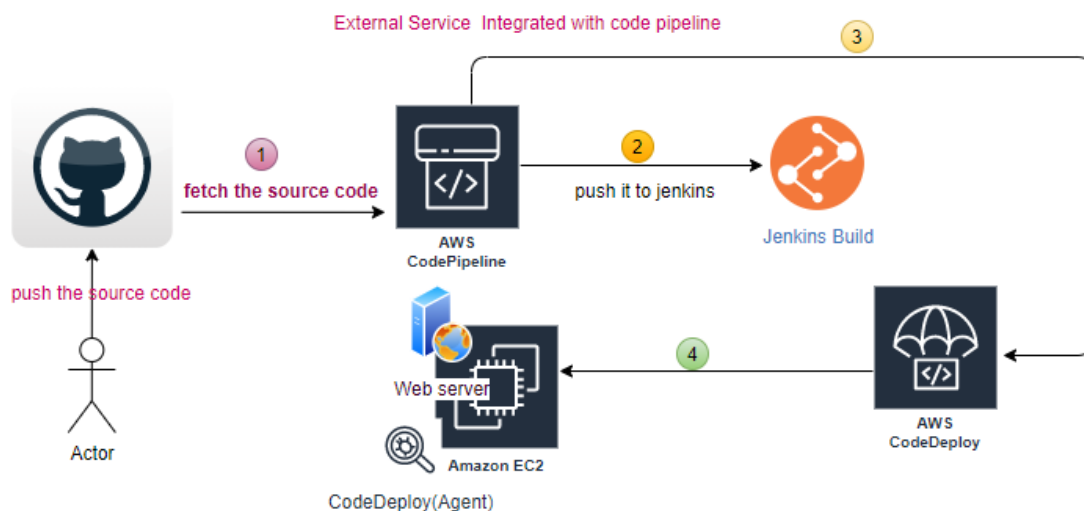
UC2: pick from s3, build it and deploy toward my webserver (EC2)



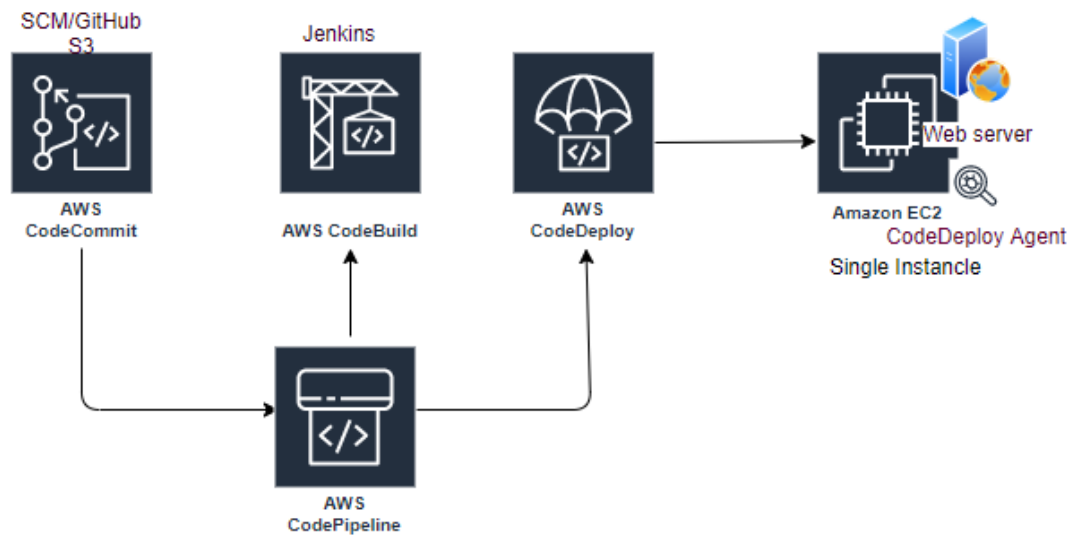
UC3: pick from CodeCommit, build it and deploy toward my webserver (EC2)



UC4: Fetch the code from Github, build in Jenkins and deploy toward my webserver (EC2)



All the above run through the CodePipeline



2:40 stuck